RetroMao / 复古猫

通过公共仓库
添加完整平台

从 Fork、图标与模型制作，到发布和应用内下载

Platform format 3 · Host 1.0.32
2026-11-05

中文完整教程 / English guide / 日本語ガイド

含可直接复制给 AI 的制作任务模板

公共仓库

适用于包含格式 3 实现的主程序构建。正式发行与真机验收状态请核对公共能力契约。

中文

1. 从哪里开始

公共仓库：Fuck996/RetroMao-Community。本教程覆盖平台资料、Logo、像素图标、包装、介质有无、GLB 模型以及发布安装。平台格式 3 的宿主门槛为 1.0.32；当前能力记录 releaseVerified=false，需使用包含本次变更的构建，正式发布和真机结果以 contracts/host.json 与发行记录为准。旧版 APK 不会因添加仓库而获得新能力。

2. Fork 并准备工具

登录 GitHub，打开上方公共仓库，右上角点 Fork，Owner 选择自己，保留公开仓库，点 Create fork。得到 https://github.com/YourAccount/RetroMao-Community。下面 YourAccount 必须替换为自己的账号；不要把上游仓库当作自己的发布目标。安装 Python 3.11+、Node.js 22+ 和 Git；发布还需要 GitHub CLI 并通过 gh auth login 登录。以下为 Windows PowerShell 命令；Linux/macOS 使用 .venv/bin/python 替换解释器路径。

```
git clone https://github.com/YourAccount/RetroMao-Community.git
cd RetroMao-Community
python -m venv .venv
.venv/Scripts/python.exe `
  -m pip install `
  --index-url https://pypi.tuna.tsinghua.edu.cn/simple `
  -r requirements-dev.txt
npm ci --registry=https://registry.npmmirror.com
```

3. 创建平台和文件目录

先使用 init 生成平台，不要从旧主工程 community 目录复制。教学示例是原创虚构机种 Demo Console，已含可打包图标和盒形模型；它不是任何真实机种的日版模型，也不带游戏和模拟器。下面复制示例素材后，用下一节完整清单替换新建的 platform.json。实际添加 MSX 等机种时，用真实资料、允许再分发的图标和日版实物参考模型替换所有教学内容。

```
.venv/Scripts/python.exe tools/retromao.py init platform `
  --id demo-console `
  --name "Demo Console" `
  --author "YourName"
Copy-Item examples/platform-presentation/assets,examples/platform-presentation/models `
  -Destination extensions/demo-console `
  -Recurse
```

文件结构

retromao-package.json 中的 id/name 必须与 platform.json 一致；填写作者、说明、三段包版本和 minAppVersion=1.0.32。该文件用于开发与发布，不进入 ZIP；RELEASE_NOTES.md 填写中英日三段实际变更和验收结果。

```
extensions/demo-console/
  platform.json
  retromao-package.json
  RELEASE_NOTES.md
  assets/logo.png
  assets/pixel.png
  models/box.glb
  models/cartridge.glb
```

4. 完整平台清单

id 为 2-32 位小写字母、数字、下划线或连字符，发布后保持稳定；名称、别名不能与内置或已安装平台冲突。extensions 不带点。coverAspectRatio 是封面宽/高，范围 0.5-2。logo 和 pixel 可分别提供透明 Logo 与像素机种图，PNG/WebP 每个最多 8 MiB、每边最多 2048。路径相对于包根目录，不允许绝对路径、反斜线或 . .。

```
{
  "schemaVersion": 3,
  "platform": {
    "id": "demo-console",
    "name": "Demo Console",
    "aliases": [
      "demo-console"
    ],
    "extensions": [
      "demo"
    ],
    "medium": "CARTRIDGE",
    "hasPackaging": true
  },
  "manufacturer": "RetroMao tutorial",
  "releaseYear": 2026,
  "coverAspectRatio": 0.7142857,
  "assets": {
    "logo": "assets/logo.png",
    "pixel": "assets/pixel.png"
  },
  "models": {
    "packaging": "models/box.glb",
    "packagingVariantLabel": "原创教学包装",
    "gameMedia": {
      "kind": "CARTRIDGE",
      "model": "models/cartridge.glb",
      "variantLabel": "原创教学卡带",
      "artworkAspectRatio": 1.25
    }
  }
}
```

5. 包装和介质有无

hasPackaging=false 表示没有包装，不能再声明包装模型；medium=NONE 表示没有实体介质，删除 models.gameMedia。两者同时成立时显示普通封面，不出现包装/介质预览与插卡/插盘音效。可只保留包装，或只保留介质。介质类型还支持 CARTRIDGE、GAME_CARD、HUCARD、DISC、UMD、DISK_CARD。有介质但暂缺模型时保留真实介质类型并省略 model，不能写 NONE 假装完成。下例是清单片段，替换对应字段，不是独立 JSON 文件。

```
"platform": {
  "id": "demo-console",
  "name": "Demo Console",
  "aliases": [
    "demo-console"
  ],
  "extensions": [
    "demo"
  ],
  "medium": "NONE",
  "hasPackaging": false
},
"models": {}
```

6. 制作模型与外壳材质

用 Blender 等工具制作并导出自包含 GLB 2.0：Y 轴向上，模型正面朝 +Z，单位与几何保持一致，应用变换并检查法线。每个 GLB 最多 32 MiB；不引用外部文件或 data URI，不使用稀疏 accessor；包最多 128 个文件、解压后 128 MiB。游戏封面/卡贴所在节点名以 artwork_ 开头，使用名称以 User artwork 开头的材质；材质的基础颜色接一张内嵌占位图片，并提供 TEXCOORD_0/UV0。宿主会用当前游戏素材替换它，缺图时仍显示内置占位图。不要把某款游戏封面烘焙到外壳。

可选换色和区域变体

需要外壳换色时，把下面 shell 放入 models.gameMedia。GLB 必须同时具备对应两种材质和 KHR_materials_variants 的 shell-opaque、shell-translucent 变体；无此结构就省略 shell。wallThickness 使用模型原单位，用于既有透色管线，不是随意填的视觉数值。裸 DISC 不支持换壳。日版默认模型放 packaging；美版可放 models.packagingVariants.US，声明 model、label 和可选 coverAspectRatio。介质标签裁切用 artworkAspectRatio，范围 0.1-10。

```
"shell": {
  "name": "Shell",
  "translucentMaterial": "Shell translucent",
  "baseColor": [
    0.2,
    0.2,
    0.2
  ],
  "roughness": 0.45,
  "wallThickness": 0.001,
  "defaultFinish": "SATIN"
}
```

7. 保留主题覆盖和启动配置

平台包提供默认图标和模型；主题只覆盖明确声明的槽位，未声明的继续使用平台包，再回到内置资源。先安装平台再安装引用它的主题。下面是已有完整 theme.json 的片段；仍需保留主题自己的 id、name、colors 等必填字段。无包装或无介质的平台不允许由主题虚构对应模型。启动配置 launchers 沿用格式 2 的 URI/RETROARCH 契约，需真实包名、完整 Activity 和经过验证的核心；包不会安装模拟器、核心或固件。不要把 MSX 的默认启动项照搬给其他机种。

```
"schemaVersion": 3,
"assets": {
  "platform.demo-console.logo": "assets/custom-logo.png",
  "model.demo-console.packaging": "models/custom-box.glb",
  "model.demo-console.gameMedia": "models/custom-cartridge.glb"
}
```

8. 校验、打包与本机验收

preview 目前展示清单和启动配置，不执行 Android Filament，也不能证明模型视觉已通过。Ctrl+C 停止预览。pack 输出 dist/platform-demo-console-1.0.0.zip 和目录条目；不要手写大小或 SHA-256。把 ZIP 保持压缩状态，在兼容构建的“设置 → 游戏库与文件夹 → 导入平台扩展包”导入，添加 ROM 目录并配置模拟器。验收 Logo/像素图、包装/介质切换、封面贴图和缺图、主题覆盖及切回默认、包更新与重启恢复。界面检查 16:9、4:3、1:1、触摸和方向键；启动/返回、固件与核心依赖用合法测试游戏和真实设备确认。

```
.venv/Scripts/python.exe tools/retromao.py check extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py preview extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py pack extensions/demo-console --release-tag demo-console-1.0.0
.venv/Scripts/python.exe tools/retromao.py check
.venv/Scripts/python.exe -m unittest discover -s tools -p "test_*.py"
npm test
```

9. 发布到自己的 GitHub 仓库

只有准备正式分享时才执行本节。Fork 不包含上游 Release 资产；不由自己分发的旧 catalog.json 条目应从自己的清单移除，或在自己的仓库发布相应资产。不要删除上游源文件来掩盖问题。先填好三语 RELEASE_NOTES.md，并确认工作区干净、origin 是自己的公开 Fork、源码提交已推到其默认分支。publish 会创建新 Release、上传 ZIP、下载回读大小与 SHA-256，成功后才改 catalog.json。再次提交推送目录后，玩家才能发现这个包。已发布版本和标签不可覆盖；修订时递增包版本和标签。注意普通手工 ZIP 打包不等于在线发布。

```
git add extensions/demo-console
git commit -m "Add demo console platform 1.0.0"
git push origin HEAD
.venv/Scripts/python.exe tools/retromao.py publish extensions/demo-console `
  --tag demo-console-1.0.0 `
  --repository YourAccount/RetroMao-Community `
  --notes extensions/demo-console/RELEASE_NOTES.md
git diff -- catalog.json
git add catalog.json
git commit -m "Publish verified demo console catalog"
git push origin HEAD
```

10. 玩家添加仓库后下载

打开复古猫“设置”首页右下角的仓库图标，每行加入一个公开 GitHub 仓库地址。官方源为 https://github.com/Fuck996/RetroMao-Community；自己的源为 https://github.com/YourAccount/RetroMao-Community。两行可以同时保留。保存后进入“设置 → 游戏库与文件夹 → 社区平台”，选择已发布且兼容的平台，下载并安装。应用从所选来源仓库的真实 Release 获取 ZIP，验证大小和摘要，然后使用与离线导入相同的安装器。完成后添加 ROM 文件夹、配置模拟器，即可在机种选择和游戏库中使用。仓库不是 WebDAV 网盘地址。

排错与更新

列表为空：核对公开仓库地址、默认分支中的 catalog.json、包类型 PLATFORM 和 minAppVersion；只上传 ZIP 而未提交目录不会出现。下载 404：确认资产在自己的 Fork 的指定 Release 中，文件名大小写一致。校验失败：保留原资产和日志，不改摘要凑通过。模型导入失败：查清单路径、GLB 内嵌资源、贴图材质命名与 UV0。更新时保持平台 ID 和启动 ID，不会因重命名自动迁移原游戏；包版本递增并先做离线更新验收。

11. 让 AI 帮你完成

把自己的 Fork 交给能读写工程的 AI，提供真实平台资料、素材授权和已验证的模拟器信息，再复制下面的提示词。AI 可以完成配置、原创图标、基于参考的建模脚本、校验和打包；无法访问设备时，模型视觉、模拟器启动和真实介质比例仍须另行验收。发布前让 AI 展示具体 diff、产物和测试结果，再明确授权 push/publish。不要把 GitHub token、模拟器密钥或 ROM 写进提示词、清单或仓库。

```
你正在我的 RetroMao-Community Fork 中工作。先读 AGENTS.md、README.md、
EXTENSIONS.md、schemas/platform.schema.json 和 docs/PLATFORM_GUIDE.md。
为我添加平台：[平台名称、稳定 ID、厂商、年份、ROM 格式]。
区域：[日版默认 / 有明确来源的区域变体]。
实体能力：[有无包装]、[介质类型或 NONE]。
图标与模型来源：[我的文件路径或获准再分发的来源、许可]。
启动方式：[已验证的模拟器包名、完整 Activity、URI 或 RetroArch 核心]。
用 init 创建 extensions/<id>，使用格式 3、最低宿主 1.0.32，
补齐 platform.json、图标、模型、发布元数据和三语发行说明。
模型必须使用可动态替换的 artwork_ 节点、User artwork 材质、
基础颜色贴图和 UV0；复用现有渲染，不把游戏封面烘焙到外壳。
没有实体介质时使用 NONE，不能用缺少模型来代替；
真实机种不能拿教学盒形冒充。素材不够时明确列出缺口。
完成校验、打包和有价值的测试，报告输出路径及真实验收结果。
没有实机时明确标记，不把预览当作 Android 验收。
先完成可审查的文件与本地提交；未经我明确授权，不推送或发布。
```

公共依据： Platform contract · Schema · Publishing · GitHub Fork guide · glTF 2.0

English

1. Start here

Public repository: Fuck996/RetroMao-Community. This guide covers identities, icons, packaging, physical-media availability, GLB models, publication and installation. Format 3 requires host 1.0.32 with this implementation. releaseVerified is currently false; consult contracts/host.json and release records for shipped/device acceptance. Adding a repository does not upgrade an older APK.

2. Fork and prepare tools

Sign in to GitHub, open the public repository, click Fork, select yourself as Owner, keep it public and click Create fork. Replace YourAccount below with your account. Your fork is the publication target. Install Python 3.11+, Node.js 22+ and Git; publishing also requires GitHub CLI and gh auth login. Commands use Windows PowerShell; use .venv/bin/python on Linux/macOS.

```
git clone https://github.com/YourAccount/RetroMao-Community.git
cd RetroMao-Community
python -m venv .venv
.venv/Scripts/python.exe `
  -m pip install `
  --index-url https://pypi.tuna.tsinghua.edu.cn/simple `
  -r requirements-dev.txt
npm ci --registry=https://registry.npmmirror.com
```

3. Create the platform files

Use init instead of copying the historical host community directory. Demo Console is fictional original tutorial artwork, not a real Japanese console replica. It contains no games or emulator. Copy the sample assets, then replace the new platform.json with the full manifest below. For a real platform, replace all tutorial identities, extensions and models with verified hardware references and redistributable assets.

```
.venv/Scripts/python.exe tools/retromao.py init platform `
  --id demo-console `
  --name "Demo Console" `
  --author "YourName"
Copy-Item examples/platform-presentation/assets,examples/platform-presentation/models `
  -Destination extensions/demo-console `
  -Recurse
```

File layout

Match id/name in retromao-package.json to platform.json. Set author, description, a three-part package version and minAppVersion=1.0.32. Metadata is excluded from the ZIP. Fill RELEASE_NOTES.md with actual changes and acceptance in Chinese, English and Japanese.

```
extensions/demo-console/
  platform.json
  retromao-package.json
  RELEASE_NOTES.md
  assets/logo.png
  assets/pixel.png
  models/box.glb
  models/cartridge.glb
```

4. Complete manifest

Use a stable 2-32-character lowercase ID with digits, underscore or hyphen. Names/aliases must not collide with existing platforms. Extensions omit the dot. coverAspectRatio is width/height, 0.5-2. logo and pixel are separate PNG/WebP roles, at most 8 MiB and 2048 pixels per side. Paths are package-relative, without absolute paths, backslashes or .. segments.

```
{
  "schemaVersion": 3,
  "platform": {
    "id": "demo-console",
    "name": "Demo Console",
    "aliases": [
      "demo-console"
    ],
    "extensions": [
      "demo"
    ],
    "medium": "CARTRIDGE",
    "hasPackaging": true
  },
  "manufacturer": "RetroMao tutorial",
  "releaseYear": 2026,
  "coverAspectRatio": 0.7142857,
  "assets": {
    "logo": "assets/logo.png",
    "pixel": "assets/pixel.png"
  },
  "models": {
    "packaging": "models/box.glb",
    "packagingVariantLabel": "原创教学包装",
    "gameMedia": {
      "kind": "CARTRIDGE",
      "model": "models/cartridge.glb",
      "variantLabel": "原创教学卡带",
      "artworkAspectRatio": 1.25
    }
  }
}
```

5. Physical availability

hasPackaging=false removes packaging and forbids packaging models. medium=NONE removes physical media; omit models.gameMedia. With neither feature, show a cover and no physical preview/insert sound. Packaging-only and media-only platforms are valid. Other media: CARTRIDGE, GAME_CARD, HUCARD, DISC, UMD, DISK_CARD. Missing geometry is not absent physical media: keep the true kind and omit model. This is a manifest fragment, not a standalone JSON file.

```
"platform": {
  "id": "demo-console",
  "name": "Demo Console",
  "aliases": [
    "demo-console"
  ],
  "extensions": [
    "demo"
  ],
  "medium": "NONE",
  "hasPackaging": false
},
"models": {}
```

6. Models and shell materials

Export self-contained GLB 2.0, Y up and front +Z, with consistent units/transforms/normals. Each GLB is at most 32 MiB; no external/data URI or sparse accessors. Archives allow 128 files and 128 MiB expanded data. Artwork nodes start artwork_, materials start User artwork, with an embedded base-color placeholder texture and TEXCOORD_0/UV0. The host replaces it with game artwork or its placeholder. Never bake one game cover into the shell.

Optional customization and variants

Put shell inside models.gameMedia only when the GLB has both named materials and KHR_materials_variants named shell-opaque/shell-translucent. Otherwise omit it. wallThickness uses model units in the existing transmission pipeline. Bare DISC models cannot customize shells. packaging is the Japanese default; models.packagingVariants.US accepts model, label and optional coverAspectRatio. artworkAspectRatio controls label cropping, 0.1-10.

```
"shell": {
  "name": "Shell",
  "translucentMaterial": "Shell translucent",
  "baseColor": [
    0.2,
    0.2,
    0.2
  ],
  "roughness": 0.45,
  "wallThickness": 0.001,
  "defaultFinish": "SATIN"
}
```

7. Theme overrides and launchers

Platform packs supply defaults. Themes override only declared slots; missing overrides keep pack defaults, then built-in assets. Install the platform before its theme. The fragment belongs in a complete theme.json with id/name/colors. Themes cannot invent absent physical features. launchers retains format-2 URI/RETROARCH contracts with real packages, full Activities and verified cores. Packs install no emulator, core or firmware; do not reuse MSX launchers for unrelated hardware.

```
"schemaVersion": 3,
"assets": {
  "platform.demo-console.logo": "assets/custom-logo.png",
  "model.demo-console.packaging": "models/custom-box.glb",
  "model.demo-console.gameMedia": "models/custom-cartridge.glb"
}
```

8. Check, pack and test

preview shows manifest/launcher data, not Android Filament. Stop with Ctrl+C. pack writes dist/platform-demo-console-1.0.0.zip and a catalog entry; never invent size/SHA-256. Import the compressed ZIP through Settings > Library and folders > Import platform extension on a compatible host; add ROM folders and configure the emulator. Test icons, physical choices, artwork/placeholders, theme override/removal, updates and restart. Check 16:9, 4:3, 1:1, touch/directional focus, and actual launch/return/core/firmware dependencies with permitted test games.

```
.venv/Scripts/python.exe tools/retromao.py check extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py preview extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py pack extensions/demo-console --release-tag demo-console-1.0.0
.venv/Scripts/python.exe tools/retromao.py check
.venv/Scripts/python.exe -m unittest discover -s tools -p "test_*.py"
npm test
```

9. Publish in your own repository

Run this only when ready to share. Forks do not inherit upstream Release assets. Remove catalog entries you do not distribute in your fork or publish their assets there; do not delete upstream source to hide the issue. Fill trilingual notes and ensure a clean tree, your public fork as origin, and source pushed to its default branch. publish creates a new Release, uploads and download-verifies ZIP size/SHA-256, then changes catalog.json. Commit/push the verified catalog so players can discover it. Versions/tags are immutable. Local packaging is not online publication.

```
git add extensions/demo-console
git commit -m "Add demo console platform 1.0.0"
git push origin HEAD
.venv/Scripts/python.exe tools/retromao.py publish extensions/demo-console `
  --tag demo-console-1.0.0 `
  --repository YourAccount/RetroMao-Community `
  --notes extensions/demo-console/RELEASE_NOTES.md
git diff -- catalog.json
git add catalog.json
git commit -m "Publish verified demo console catalog"
git push origin HEAD
```

10. Players add and download

Open the repository icon at the lower right of Settings. Add one public GitHub repository URL per line: https://github.com/Fuck996/RetroMao-Community and/or https://github.com/YourAccount/RetroMao-Community. Save, open Settings > Library and folders > Community platforms, select a published compatible pack and download/install. The app obtains the source repository Release ZIP, verifies size/hash and uses the offline installer. Then add ROM folders and configure an emulator. Repository URLs are not WebDAV connections.

Troubleshooting and updates

Empty list: check public URL, default-branch catalog.json, PLATFORM kind and minAppVersion. ZIP upload alone is insufficient. A 404 usually needs checking the fork Release/tag and exact filename. Preserve logs/assets on hash failure; do not change hashes to bypass it. Model rejection: inspect relative paths, embedded resources, material names and UV0. Keep platform/launcher IDs stable on updates, increment package version and test offline updates first.

11. Ask AI to build it

Give a repository-capable AI your fork, verified hardware facts, asset licenses and emulator information, then use the prompt below. AI can author configuration, original icons, modeling scripts, checks and packaging. Device visuals and emulator behavior still need actual acceptance. Review the diff, artifacts and results before authorizing push/publish. Never put tokens, emulator secrets or ROMs into prompts/manifests/repositories.

```
Work in my RetroMao-Community fork. Read AGENTS.md, README.md,
EXTENSIONS.md, schemas/platform.schema.json and docs/PLATFORM_GUIDE.md first.
Add [name, stable ID, manufacturer, year, ROM extensions]. Region: [Japanese
default / documented variant]. Physical features: [packaging yes/no],
[medium type or NONE]. Licensed assets: [paths, sources and licenses].
Launch: [verified package, full Activity, URI or RetroArch cores].
Run init; author format 3 with minAppVersion 1.0.32 under extensions/<id>.
Include manifest, icons, models, metadata and trilingual release notes.
Use artwork_ nodes, User artwork materials, base-color texture and UV0.
Reuse rendering. Do not bake game covers into shells or pass tutorial
geometry off as real hardware. Declare NONE for absent physical media.
Report missing assets, run validation/pack/tests and report actual results.
Preview is not Android acceptance. Commit locally for review;
do not push or publish without my explicit authorization.
```

References: Platform contract · Schema · Publishing · GitHub Fork guide · glTF 2.0

日本語

1. 開始する前に

公開リポジトリ：Fuck996/RetroMao-Community。機種情報、アイコン、箱、媒体の有無、GLB、公開と導入を説明します。形式 3 には本実装を含むホスト 1.0.32 が必要です。現在 releaseVerified=false であり、正式公開と実機結果は contracts/host.json と公開記録で確認します。リポジトリ登録だけでは旧 APK の機能は増えません。

2. Fork と環境の準備

GitHub にログインし、公開元の Fork を押し、Owner に自分を選び、公開のまま Create fork を実行します。以下の YourAccount を自分のアカウントに置き換えます。公開先は自分の Fork です。Python 3.11+、Node.js 22+、Git が必要で、公開には GitHub CLI と gh auth login も必要です。例は Windows PowerShell 用です。Linux/macOS では .venv/bin/python を使用します。

```
git clone https://github.com/YourAccount/RetroMao-Community.git
cd RetroMao-Community
python -m venv .venv
.venv/Scripts/python.exe `
  -m pip install `
  --index-url https://pypi.tuna.tsinghua.edu.cn/simple `
  -r requirements-dev.txt
npm ci --registry=https://registry.npmmirror.com
```

3. 機種ファイルを作成

旧ホストの community ではなく init から作成します。Demo Console は架空の教材で、実機の日本版を再現したものではありません。ゲームとエミュレーターは含みません。素材をコピーし、次節の設定で platform.json を置き換えます。実機を追加する場合は名称、拡張子、素材を確認済みの情報と再配布可能な日本版モデルに交換してください。

```
.venv/Scripts/python.exe tools/retromao.py init platform `
  --id demo-console `
  --name "Demo Console" `
  --author "YourName"
Copy-Item examples/platform-presentation/assets,examples/platform-presentation/models `
  -Destination extensions/demo-console `
  -Recurse
```

ファイル構成

retromao-package.json の id/name を platform.json と一致させ、作者、説明、三桁の版と minAppVersion=1.0.32 を設定します。この情報は ZIP に入りません。RELEASE_NOTES.md の三言語に実際の変更と検証結果を記載します。

```
extensions/demo-console/
  platform.json
  retromao-package.json
  RELEASE_NOTES.md
  assets/logo.png
  assets/pixel.png
  models/box.glb
  models/cartridge.glb
```

4. 完全な設定例

ID は小文字英数字、アンダースコア、ハイフンの 2-32 文字で、公開後は固定します。既存の名称と別名を重複させません。拡張子に点は不要です。coverAspectRatio は幅/高さで 0.5-2。logo と pixel は別々の PNG/WebP で、各 8 MiB、各辺 2048 ピクセルまで。パスは相対で、絶対パス、バックスラッシュ、.. は使えません。

```
{
  "schemaVersion": 3,
  "platform": {
    "id": "demo-console",
    "name": "Demo Console",
    "aliases": [
      "demo-console"
    ],
    "extensions": [
      "demo"
    ],
    "medium": "CARTRIDGE",
    "hasPackaging": true
  },
  "manufacturer": "RetroMao tutorial",
  "releaseYear": 2026,
  "coverAspectRatio": 0.7142857,
  "assets": {
    "logo": "assets/logo.png",
    "pixel": "assets/pixel.png"
  },
  "models": {
    "packaging": "models/box.glb",
    "packagingVariantLabel": "原创教学包装",
    "gameMedia": {
      "kind": "CARTRIDGE",
      "model": "models/cartridge.glb",
      "variantLabel": "原创教学卡带",
      "artworkAspectRatio": 1.25
    }
  }
}
```

5. 箱と媒体の有無

hasPackaging=false は箱なしで、箱モデルは指定できません。medium=NONE は実物媒体なしで、models.gameMedia を省きます。両方がない場合は表紙を表示し、実物プレビューと挿入音は使いません。箱のみ、媒体のみも可能です。他の媒体は CARTRIDGE、GAME_CARD、HUCARD、DISC、UMD、DISK_CARD。モデル未完成を NONE で代用せず、正しい種類のまま model を省きます。以下は設定の一部分です。

```
"platform": {
  "id": "demo-console",
  "name": "Demo Console",
  "aliases": [
    "demo-console"
  ],
  "extensions": [
    "demo"
  ],
  "medium": "NONE",
  "hasPackaging": false
},
"models": {}
```

6. モデルと外装材質

自己完結した GLB 2.0 を出力します。Y が上、正面は +Z とし、単位、変換、法線を確認します。GLB は各 32 MiB、外部/data URI と sparse accessor は不可。パックは 128 ファイル、展開後 128 MiB まで。画像面のノード名は artwork_、材質名は User artwork で始め、内蔵の基本色画像と TEXCOORD_0/UV0 を設定します。実行時はゲーム画像または標準の代替画像に置き換わります。外装に特定ゲームの表紙を焼き込まないでください。

任意の色変更と地域版

色変更が必要な場合だけ models.gameMedia に shell を追加します。指定した二つの材質と KHR_materials_variants の shell-opaque / shell-translucent が必要です。なければ shell を省略します。wallThickness はモデル単位の実厚で、既存の透過処理に使います。裸 DISC の外装変更は不可。日本版既定は packaging、美版は models.packagingVariants.US に model、label、任意の coverAspectRatio を指定します。ラベル比率 artworkAspectRatio は 0.1-10 です。

```
"shell": {
  "name": "Shell",
  "translucentMaterial": "Shell translucent",
  "baseColor": [
    0.2,
    0.2,
    0.2
  ],
  "roughness": 0.45,
  "wallThickness": 0.001,
  "defaultFinish": "SATIN"
}
```

7. テーマ上書きと起動

パックが既定を提供し、テーマは指定したスロットだけ上書きします。未指定はパック、次に内蔵素材を使います。先に機種を導入します。例は完全な theme.json の一部で、id/name/colors は別途必要です。存在しない箱や媒体はテーマで追加できません。launchers は形式 2 の URI/RETROARCH を継承し、実在のパッケージ、完全な Activity、確認済みコアを使います。エミュレーター、コア、ファームウェアは同梱しません。MSX の起動定義を他機種へ流用しないでください。

```
"schemaVersion": 3,
"assets": {
  "platform.demo-console.logo": "assets/custom-logo.png",
  "model.demo-console.packaging": "models/custom-box.glb",
  "model.demo-console.gameMedia": "models/custom-cartridge.glb"
}
```

8. 検査、パッケージ化と検証

preview は設定と起動一覧だけで、Android Filament は実行しません。Ctrl+C で停止します。pack は dist/platform-demo-console-1.0.0.zip と索引項目を生成します。サイズと SHA-256 を捏造しません。対応ホストの設定 > ライブラリーとフォルダー > 機種拡張インポートで ZIP のまま導入し、ROM フォルダーとエミュレーターを設定します。アイコン、箱/媒体、画像と代替画像、テーマ上書きと解除、更新と再起動を確認します。16:9、4:3、1:1、タッチ、方向キー、実機の起動/復帰とコア/ファームウェア依存も検証します。

```
.venv/Scripts/python.exe tools/retromao.py check extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py preview extensions/demo-console
.venv/Scripts/python.exe tools/retromao.py pack extensions/demo-console --release-tag demo-console-1.0.0
.venv/Scripts/python.exe tools/retromao.py check
.venv/Scripts/python.exe -m unittest discover -s tools -p "test_*.py"
npm test
```

9. 自分のリポジトリで公開

共有する準備ができた場合だけ実行します。Fork は上流の Release を継承しません。自分が配布しないカタログ項目は自分の索引から除くか、必要なアセットを自分で公開します。元ソースは削除しません。三言語の説明を記入し、作業ツリーをクリーンにし、origin と公開 Fork を一致させ、ソースを既定ブランチに push します。publish は新規 Release、ZIP アップロード、再取得のサイズ/ハッシュ検証後に catalog.json を変更します。その索引もコミットして push すると検出されます。既存版とタグは上書きできません。ローカル ZIP はオンライン公開とは異なります。

```
git add extensions/demo-console
git commit -m "Add demo console platform 1.0.0"
git push origin HEAD
.venv/Scripts/python.exe tools/retromao.py publish extensions/demo-console `
  --tag demo-console-1.0.0 `
  --repository YourAccount/RetroMao-Community `
  --notes extensions/demo-console/RELEASE_NOTES.md
git diff -- catalog.json
git add catalog.json
git commit -m "Publish verified demo console catalog"
git push origin HEAD
```

10. 利用者が登録してダウンロード

設定ホーム右下のリポジトリアイコンを開き、公開 GitHub URL を一行ずつ追加します。公式は https://github.com/Fuck996/RetroMao-Community、自分の配布元は https://github.com/YourAccount/RetroMao-Community です。両方登録できます。保存後、設定 > ライブラリーとフォルダー > コミュニティ機種で公開済みの互換パックを選び、導入します。アプリは公開元 Release の ZIP のサイズとハッシュを検証し、オフラインと同じ処理で導入します。その後 ROM とエミュレーターを設定します。WebDAV の URL ではありません。

問題の確認と更新

一覧が空なら公開 URL、既定ブランチの catalog.json、PLATFORM 種別、minAppVersion を確認します。ZIP のアップロードだけでは表示されません。404 は自分の Release、タグ、ファイル名を確認します。ハッシュ失敗時はログを残し、回避のため値を変更しません。モデル拒否は相対パス、内蔵資源、材質名、UV0 を確認します。更新では機種/起動 ID を維持し、パック版を上げ、先にオフライン更新を検証します。

11. AI に作業を依頼

リポジトリを操作できる AI に自分の Fork と確認済みの実機情報、素材の許可、エミュレーター情報を渡し、以下を使います。AI は設定、オリジナル素材、モデリングスクリプト、検査とパッケージ化を行えます。実機の見た目と起動は別途検証します。差分、成果物と結果を確認してから push/publish を許可してください。token、秘密情報、ROM をプロンプトやリポジトリに入れません。

```
私の RetroMao-Community Fork 内で作業してください。最初に AGENTS.md、
README.md、EXTENSIONS.md、schemas/platform.schema.json、
docs/PLATFORM_GUIDE.md を読んでください。
追加する機種：[名称、固定 ID、メーカー、発売年、ROM 拡張子]。
地域：[日本版を既定 / 出典の明確な地域版]。
実物：[箱の有無]、[媒体の種類または NONE]。
素材：[ファイル、配布可能な出典とライセンス]。
起動：[検証済みのパッケージ、完全な Activity、URI またはコア]。
init を使用し、extensions/<id> に形式 3、最低 1.0.32 のパックを作成。
設定、アイコン、モデル、公開メタデータと三言語の説明を含めます。
artwork_ ノード、User artwork 材質、基本色テクスチャと UV0 を使用。
既存の描画を使い、実機に教材の形状を流用しないでください。
媒体なしは NONE と宣言します。不足素材と実際の検証結果を報告し、
検査、パッケージ化、テストを完了してください。
プレビューを Android 実機検証と見なさないでください。
レビュー用のローカルコミットまで行い、許可なく push や公開しない。
```

参照： Platform contract · Schema · Publishing · GitHub Fork guide · glTF 2.0